Lessons Learned

Simulated retrospective synthesis and actions for future projects

Document	15 - Lessons Learned
Project	AI Customer Support Platform (fictional company: Kestrel Commerce Ltd.)
Prepared by	Project Manager (portfolio author) - self-directed simulation
Status	Simulated scenario
Project window	5 Jan - 13 Mar 2026 (simulated project window)

SIMULATED PROJECT - Self-directed portfolio case study. Not professional experience, not client work, not employment. All people, figures and outcomes are fictional.

Simulated lessons learned. In a real project these would be collected from team retrospectives; here they are a synthesised retrospective for the simulated scenario and show the kind of reflection and action-setting a PM would lead.

1. Lessons and actions

Area	What happened (scenario)	Lesson	Action for next project	Owner
Planning	Sprint 1 delivered 25 of 28 points; the new team had no velocity history.	Plan the first sprint lighter and re-forecast from actual velocity after two sprints.	Apply a 10-15% ramp-up discount to a new team's first sprint.	Project Manager
Vendors / legal	LLM provider data-handling terms were still unconfirmed at the end of Sprint 2.	Vendor and privacy reviews take calendar time that the team cannot speed up.	Open the vendor/legal review in Week 1 and track it as a go-live gate dependency from day one.	Project Manager
Content	Knowledge-base audit found 41 duplicates and 52 outdated articles out of 320.	Answer quality is capped by content quality; the audit was worth starting in Week 2.	Name a content owner per category in the charter and start the audit at kickoff.	Product Manager
AI quality	Golden-set reference answers were validated late (72 of 100 at 30 Jan).	SME time is a dependency like any other.	Book SME hours with dates at kickoff and report them in the dependency log.	QA Engineer
AI quality	Putting the evaluation score in the Definition of Done exposed a regression after a prompt change in Sprint 3.	Quality gates work only if they run every sprint, not at the end.	Keep a no-drop-over-2-points rule for AI stories.	AI/ML Engineer
Scope	CR-001 (Spanish) and the helpdesk integration request were declined without conflict.	A published Won't-Have list and a simple change form depersonalise 'no, not now'.	Publish the Won't-Have list at the requirements baseline.	Project Manager
Estimation	Velocity stabilised at 34-37 points after Sprint 2.	Early velocity is a poor predictor; tuning and ramp-up effects fade.	Do not extrapolate from one sprint; re-plan Sprint 5 only after Sprint 4.	Product Manager
Integration	Contract-first API definitions and a staging environment from Week 1 avoided late integration rework (R10 did not materialise).	Early integration lowers risk more than late testing.	Keep OpenAPI contracts agreed before UI work.	Backend Developer
Testing	The load test sat in Sprint 5 and left three working days to react.	Late performance tests leave little room to fix.	Add a smoke test one sprint before the formal load test.	QA Engineer
Scope / security	Audit logging (Should, security-related) was planned for Sprint 5 and deferred when UAT fixes used capacity.	Items planned last are cut first, whatever their value.	Schedule security-relevant Should items before the final sprint.	Product Manager
Stakeholders	Asking for UAT participants five weeks ahead meant they were available.	Early, dated requests beat urgent ones.	Book UAT and training slots at the start of Sprint 3.	Project Manager
Communication	A one-page status with a 'decisions required' table produced decisions within two days.	Executives respond to clear options with a recommendation.	Always pair a problem with options and a recommended choice.	Project Manager

2. What went well

- Grounding, citations and a visible fallback were designed in from Sprint 2 and built stakeholder trust.
- Change control and a published Won't-Have list kept the Must-Have scope stable.
- Evaluation results were shown at every sprint review, which kept accuracy expectations realistic (R15).
- A risk register with triggers and owners turned discussions into specific actions.
- Capacity was planned with a visible 10% buffer.

3. What could be improved

- Treat vendor terms, SME time and DNS changes as scheduled dependencies from Week 1.
- Plan security-related Should-Have work earlier in the schedule.
- Run performance smoke tests earlier.
- Capture decisions in a single decision log from the first meeting (it was introduced in Sprint 1 but not enforced until Sprint 2).

4. Retrospective themes by sprint (scenario)

Sprint	Keep	Change
1	Daily visibility of blockers; early staging environment.	Clarify the permission matrix before estimating RBAC.
2	Demo of real answers with sources.	Book SMEs for golden-set validation in advance.
3	Evaluation gate in the Definition of Done.	Re-forecast the next sprint immediately after each review.
4	Agent walkthrough of the console before UAT.	Schedule load smoke test earlier.
5	Hypercare checklist.	Protect Should-Have items from late cuts, or drop them explicitly at planning.

5. Limitations

Lessons are plausible, scenario-based reflections and are not drawn from a real team. They are written to be discussed honestly in an interview as what a PM would aim to learn and change.